ISIMA - Clermont Auvergne INP

Programmation orientée objet
Z225DM04
Rapport

Démineur

Élèves :
Valentin Portail
Jérémie Villepreux

Enseignant :
Yannick Loiseau
Youssef Bouaziz

16 avril 2023

Table des matières

1 Comment lancer notre projet ? 2

2 Choix d'implémentation 2

2.1 Une vue d'ensemble de notre projet 2
2.2 Justification du choix de la structure générale 3
2.3 Retour en détail sur quelques classes 3
2.3.1 La grille 3
2.3.2 Les cellules 3
2.3.3 La sauvegarde et chargement des parties 4
2.3.4 Le mode d'affichage 4

A Diagramme UML 5

1 Comment lancer notre projet ?

Pour exécuter notre programme, il faut se placer dans le répertoire principal de notre projet, puis lancer la commande :

```
$ javac -d build src/*.java
```

En cas de problème, les fichiers sont déjà pré-compilés dans le dossier `build`.

L'exécution de notre projet nécessite *obligatoirement* de se placer dans le répertoire `build` et de lancer la commande :

```
$ java Main
```

Se positionner dans le répertoire `build` est obligatoire en raison d'un aspect sur la sauvegarde des fichiers, le chemin est donnée relativement par rapport à ce dossier pour le chargement d'une grille qui a été sauvegarder.

2 Choix d'implémentation

2.1 Une vue d'ensemble de notre projet

Avant d'entrer en détail sur quelques aspects de l'implémentation de nos classes, il nous a paru utile de donner une approche globale du projet. Il est possible de se référer ainsi à la figure 1 pour avoir cette vue générale. Nous avons divisé notre implémentation du démineur en douze classes :

- Cell : classe abstraite qui contient différents attributs et méthodes afin de représenter au mieux une case ;
- FreeCell : classe qui représente concrètement une case non-minée ;
- MineCell : classe qui représente concrètement une case minée ;
- Status : classe définissant seulement un statut (caché, marqué, découvert), afin de garantir un état à la case ;
- Direction : énumération de 8 directions. Elle servira dans une EnumMap à connaître les 8 voisins d'une case. En effet, chaque case a pour voisins 8 autres cases (si elles existent) de manière doublement chaînée. Nous avons fait le choix d'énumérer les points cardinaux, et donc de pouvoir passer d'une direction à l'autre (dans le sens horaire), par incrémentation de celle-ci ;
- Grid : classe définissant la grille de jeu. Elle n'utilise pas de tableau ;
- Game : classe qui permet, lorsqu'elle a été correctement initialisée, de lancer une partie de démineur ;
- GridFileManager : classe qui permet la gestion de la grille par sauvegarde et chargement depuis un fichier ;
- Main : simple classe qui permet de créer un point d'entrée à notre programme et de démarrer une partie ;
- UI : interface contenant des méthodes nécessaires pour pouvoir interagir avec l'utilisateur ;
- TUI : classe implémentant une interface utilisateur textuelle, dans le terminal ;
- GUI : classe implémentant une interface utilisateur graphique, dans une fenêtre spécifique, avec la bibliothèque Swing ;

2.2 Justification du choix de la structure générale

Nous avons fait le choix d'une structure plutôt simple, mais robuste pour le démineur. En plus des principes de bases de la programmation orientée objet comme l'encapsulation (par la mise en place d'accesseurs et de délégation), nous avons mis en place de l'héritage. Grâce à cette approche, on peut distinguer de manière précise si une cellule est minée ou non, en regardant le type de la cellule. Cela est aussi plus sécurisant, car au lieu de regarder un booléen facilement modifiable, il suffit de comparer les types pour savoir si une cellule est minée ou non.

Pour un détail explicite des méthodes, il est possible de se référencer directement aux commentaires dans le code.

2.3 Retour en détail sur quelques classes

2.3.1 La grille

Une instance de grille possède une largeur (`height`), une longueur (`width`), et une cellule d'origine (`gridStart`). Les cellules ne sont *pas* stockées dans un tableau. Pour réaliser le chaînage, on part de la cellule `gridStart`, puis on lui rajoute un voisin à droite. À ce voisin de droite, on lui rajoute un nouveau voisin de droite, etc. Une fois que l'on a ajouté `height` cellules, on dit qu'on a fini notre ligne. On passe alors à la ligne suivante. On refait le même processus en garantissant le double chaînage à n'importe quel moment, et ce dans les 8 directions. Le processus s'arrête lorsque l'on a ajouté le bon nombre de cellules, c'est-à-dire lorsque l'on à complété les `height` colonnes et les `width` lignes. C'est la fonction `buildGrid` qui garantit la bonne construction de la grille. En prenant soin de mettre les cellules de types `FreeCell` ou `MineCell` au besoin. La méthode `buildGrid` peut éventuellement prendre en paramètre un tableau contenant les coordonnées des mines, ce qui sera pratique pour charger une grille depuis un fichier. Si aucun tableau n'est spécifié, la méthode créera automatiquement un nouveau tableau de mines en fonction du pourcentage de mines voulu. Cette variante sera utilisée pour créer une grille à partir de zéro.

Cette approche, qui n'utilise pas de matrice, se révèle utile car notre grille ne dépend pas d'un tableau. Ainsi, on pourrait créer des grilles ayant différentes formes. Il devient ainsi possible de définir une grille en forme de cercle ou de smiley pour jouer au démineur. C'est cette approche qui nous a été fortement recommandée par notre chargé de TD. Néanmoins, cette méthode ne nous permet pas d'accéder à une cellule en temps constant, car il faut toujours partir de `gridStart` pour accéder à une cellule. Cela est donc un compromis à faire.

2.3.2 Les cellules

Nous avons stocké les voisins de chaque cellule dans un `EnumMap`. C'est aussi dans la classe `Cell` que se trouvent les méthodes pour faire le chaînage entre les cellules. Cela permet de faire de délégation. De plus, chaque cellule non minée possède également le nombre de cellules dans son voisinage direct qui sont minées. Nous avons aussi redéfini `toString`, pour afficher un contenu personnalisé selon le statut de la case. La méthode `uncoverCell` a aussi été redéfinie. C'est pour cette raison que la classe `Cell` est *abstraite*. Une cellule est donc forcément soit minée, soit libre.

2.3.3 La sauvegarde et chargement des parties

La classe `GridFileManager` a été rajoutée pour favoriser l'encapsulation. Les méthodes de cette classe auraient pu directement être mises dans la classe `Grid`, mais cela va à l'encontre des principes de l'objet en plus d'alourdir le code. Cette classe a pour but de gérer la sauvegarde des grilles. Un fichier de sauvegarde de grille a une extension `txt`. Elles ont toutes les mêmes structures :

- la première ligne contient le nombre de colonnes de la grille ;
- la deuxième ligne contient le nombre de lignes de la grille ;
- la troisième ligne contient le nombre de mines dans la grille ;
- si la cellule est une mine, elle sera indiquée par la lettre `M`, suivie du statut de la case : `H` (cachée), `U` (découverte), `F` (marquée d'un drapeau) ;
- si la cellule est une case non minée, elle sera indiquée par un numéro, indiquant le nombre de mines dans son voisinage, suivi du statut de la case : `H` (cachée), `U` (découverte), `F` (marquée d'un drapeau) ;
- un espace est ajouté entre chaque case afin de faciliter le parsing ;
- un retour à la ligne est ajouté en fin de ligne afin de faciliter également le parsing ;

2.3.4 Le mode d'affichage

L'interface `UI` contient les méthodes nécessaires pour pouvoir interagir avec l'utilisateur. Elle permet de rajouter un niveau d'abstraction afin de pouvoir définir une « interface » commune entre les différentes interfaces de notre jeu. Ainsi, on peut réunir différentes implémentations, et ce de manière indépendante de leurs types d'implémentation (`Swing`, `lanterna`, terminal...).

2.3.4.1 Le mode terminal Le code du mode terminal est situé dans la classe `TUI`. Il définit l'implémentation de base pour jouer dans le terminal. Il définit les méthodes de l'interface `UI` et rajoute celles propres au terminal. Il est composé uniquement des méthodes permettant une interaction entre le jeu et l'utilisateur. Il contient donc les méthodes pour demander la taille de la grille, le pourcentage de mines, le choix de la case, l'action à effectuer... C'est-à-dire toute interaction (entrée/sortie) avec l'utilisateur.

2.3.4.2 Le mode graphique Le code du mode graphique est situé dans la classe `GUI`. Il définit l'implémentation de base pour jouer avec une interface `Swing`. Il repose sur l'interface `UI`. Il est lui aussi composé des méthodes permettant une interaction avec l'utilisateur, comme précédemment. Il contient aussi toutes les méthodes dites *événementielles*, où l'action d'un utilisateur est attendue afin de réaliser une action. Une classe `GUICell` a également été définie à l'intérieur de la classe `GUI` afin d'associer à chaque cellule une position (dans le but de la placer dans la grille affichée à l'écran). Dans la grille de jeu, nous avons choisi :

- * : pour représenter une bombe ;
- P : pour représenter un drapeau posé ;

Au début, l'utilisation de certains pictogrammes présents dans le standard Unicode pour représenter les drapeaux et les bombes a été envisagée, mais dû à des problèmes de compatibilité entre les appareils, on a préféré se restreindre aux caractères ASCII.

A Diagramme UML

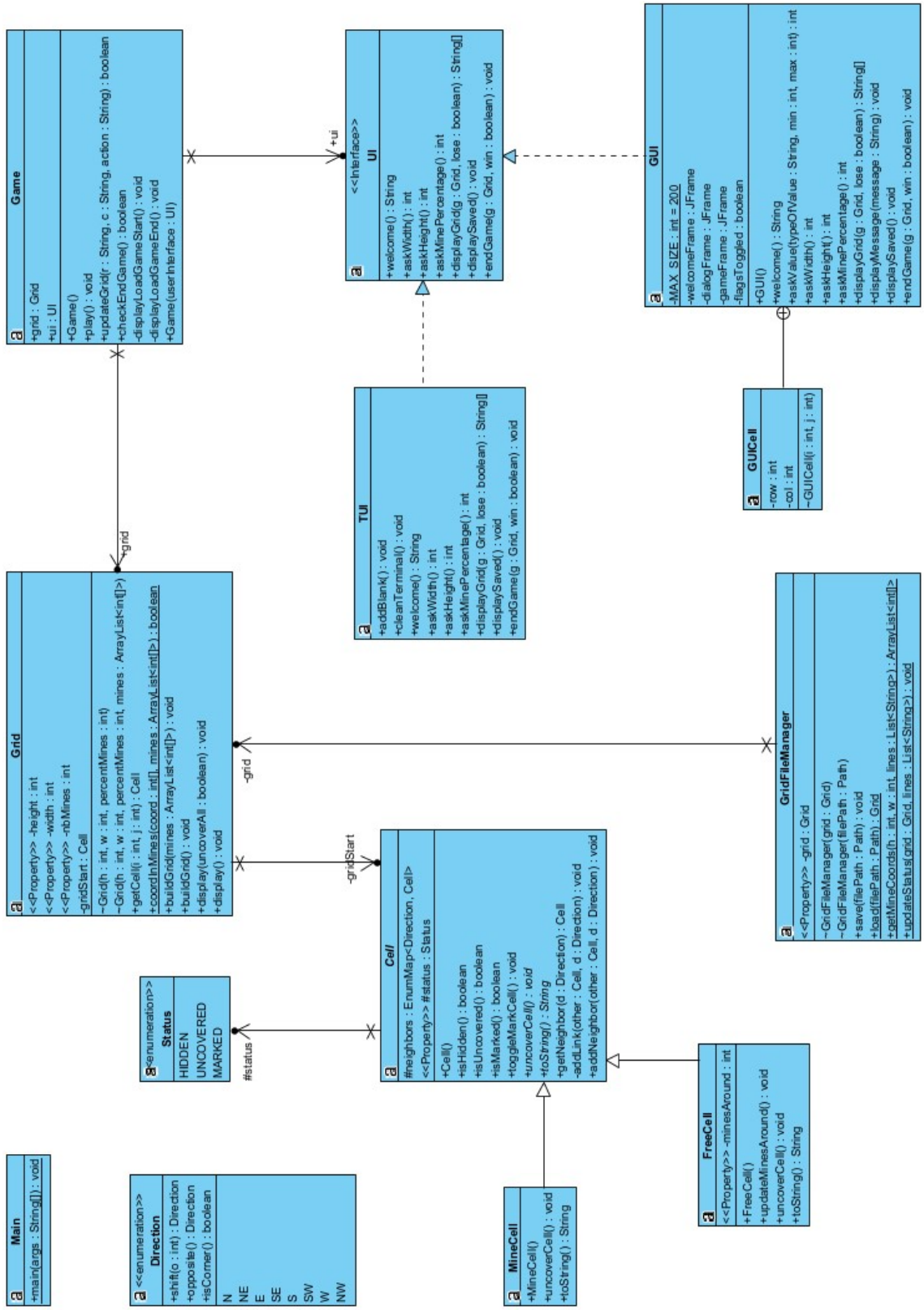

FIGURE 1 – Diagramme UML de l'ensemble de notre projet